NAMA: BUNGA BUDI AMBAR WATI

NIM: H1D024006

SHIFT KRS: G

SHIFT SEKARANG: F

PENJELASAN SOURCE CODE PRAKTIKUM KECERDASAN BUATAN PERT 3

STUDY KASUS 1

SOURCE CODE & PENJELASAN

- Import

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

Bagian import ini berfungsi sebagai:

a) numpy → buat array/rentang nilai
b) skfuzzy → buat logika fuzzy & membership function
c) control (ctrl) → bikin sistem fuzzy & rule
d) matplotlib → nampilin grafik
e) warnings → nyembunyiin warning biar output rapi

- Variabel

```
# 1. Variabel

terjual = ctrl.Antecedent(np.arange(0, 101, 1), 'Barang Terjual')
permintaan = ctrl.Antecedent(np.arange(0, 301, 1), 'Permintaan')
harga = ctrl.Antecedent(np.arange(0, 100001, 1), 'Harga per Item')
profit = ctrl.Antecedent(np.arange(0, 4000001, 1), 'Profit')
stok = ctrl.Consequent(np.arange(0, 1001, 1), 'Stok Makanan')
```

Mendefinisikan semua variable yang digunakan:

Input: barang terjual, permintaan, harga, profit

Output: stok makanan

Setiap variabel punya rentang nilai (misal 0–100, 0–300, dll) supaya sistem tahu batas data yang diproses. karena semua perhitungan fuzzy terjadi di dalam range tersebut.

- Membership Function

```
# 2. Membership Function

terjual['rendah'] = fuzz.trimf(terjual.universe, [0, 0, 40])
terjual['sedang'] = fuzz.trimf(terjual.universe, [30, 50, 70])
terjual['tinggi'] = fuzz.trimf(terjual.universe, [60, 100, 100])

permintaan['rendah'] = fuzz.trimf(permintaan.universe, [0, 0, 100])
permintaan['sedang'] = fuzz.trimf(permintaan.universe, [50, 150, 250])
permintaan['tinggi'] = fuzz.trimf(permintaan.universe, [200, 300, 300])

harga['murah'] = fuzz.trimf(harga.universe, [0, 0, 40000])
harga['sedang'] = fuzz.trimf(harga.universe, [30000, 50000, 80000])
harga['mahal'] = fuzz.trimf(harga.universe, [60000, 100000, 100000])

profit['rendah'] = fuzz.trimf(profit.universe, [0, 0, 1500000])
profit['sedang'] = fuzz.trimf(profit.universe, [1000000, 2000000, 2500000])
profit['tinggi'] = fuzz.trapmf(profit.universe, [1500000, 2500000, 4000000,
4000000])

stok['sedang'] = fuzz.trimf(stok.universe, [100, 500, 900])
stok['banyak'] = fuzz.trimf(stok.universe, [600, 1000, 1000])
```

tiap variabel dibagi jadi kategori: rendah , sedang, tinggi.

Sebagian besar pakai segitiga (trimf) → sederhana dan cukup untuk representasi data.
Khusus profit tinggi pakai trapesium (trapmf) → supaya nilai tinggi lebih stabil (tidak langsung turun).

- Visualisasi

```
# 3. Visualisasi (FIX)

def tampilkan(var, judul):
    # .view() otomatis membuat figure baru, tidak perlu ditimpa subplots() yang
kosong
    var.view()
    plt.title(judul)

tampilkan(terjual, "Barang Terjual")
tampilkan(permintaan, "Permintaan")
```

```
tampilkan(harga, "Harga per Item")
tampilkan(profit, "Profit")
tampilkan(stok, "Stok Makanan")
```
var.view()

Tujuannya:

- memastikan bentuk kurva benar
- melihat pembagian kategori secara visual

Grafik baru muncul setelah `plt.show()` di akhir.

- Rules

```
# 4. Rules

rule1 = ctrl.Rule(terjual['tinggi'] & permintaan['tinggi'] & harga['murah'] &
profit['tinggi'], stok['banyak'])
rule2 = ctrl.Rule(terjual['tinggi'] & permintaan['tinggi'] & harga['murah'] &
profit['sedang'], stok['sedang'])
rule3 = ctrl.Rule(terjual['tinggi'] & permintaan['sedang'] & harga['murah'] &
profit['sedang'], stok['sedang'])
rule4 = ctrl.Rule(terjual['sedang'] & permintaan['tinggi'] & harga['murah'] &
profit['sedang'], stok['sedang'])
rule5 = ctrl.Rule(terjual['sedang'] & permintaan['tinggi'] & harga['murah'] &
profit['tinggi'], stok['banyak'])
rule6 = ctrl.Rule(terjual['rendah'] & permintaan['rendah'] & harga['sedang'] &
profit['sedang'], stok['sedang'])
```

Total ada **6 aturan**, yang mewakili kondisi nyata:

- kondisi bagus → stok banyak
- kondisi biasa → stok sedang

tiap aturan menghubungkan kondisi input dengan keputusan stok

- Sistem

```
# 5. Sistem

stok_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6])
simulasi = ctrl.ControlSystemSimulation(stok_ctrl)
```

Digunakan untuk:

- menggabungkan semua rule
- menyiapkan proses perhitungan

Ibaratnya ini mesin utama yang akan jalanin semua logika fuzzy.

- Input

```
# 6. Input

simulasi.input['Barang Terjual'] = 80
simulasi.input['Permintaan'] = 255
simulasi.input['Harga per Item'] = 25000
simulasi.input['Profit'] = 3500000
```

Data yang dimasukkan:

- terjual = 80
- permintaan = 255
- harga = 25.000
- profit = 3.500.000

Data ini nanti diolah oleh sistem fuzzy.

- Proses (Compute)

```
# 7. Proses

simulasi.compute()
```

Di dalamnya ada 3 tahap:

a) Fuzzifikasi → ubah angka jadi derajat keanggotaan
b) Inferensi → jalankan semua rule
c) Defuzzifikasi → hasil akhir jadi angka pasti

Semua proses ini berjalan otomatis.

- Output & Visualisasi Akhir

```
# 8. Output

hasil = simulasi.output['Stok Makanan']
print("Jumlah stok optimal:", hasil)

plt.show()
```

Di dalamnya ada 3 tahap:

a) Fuzzifikasi → ubah angka jadi derajat keanggotaan
b) Inferensi → jalankan semua rule
c) Defuzzifikasi → hasil akhir jadi angka pasti

Semua proses ini berjalan otomatis.

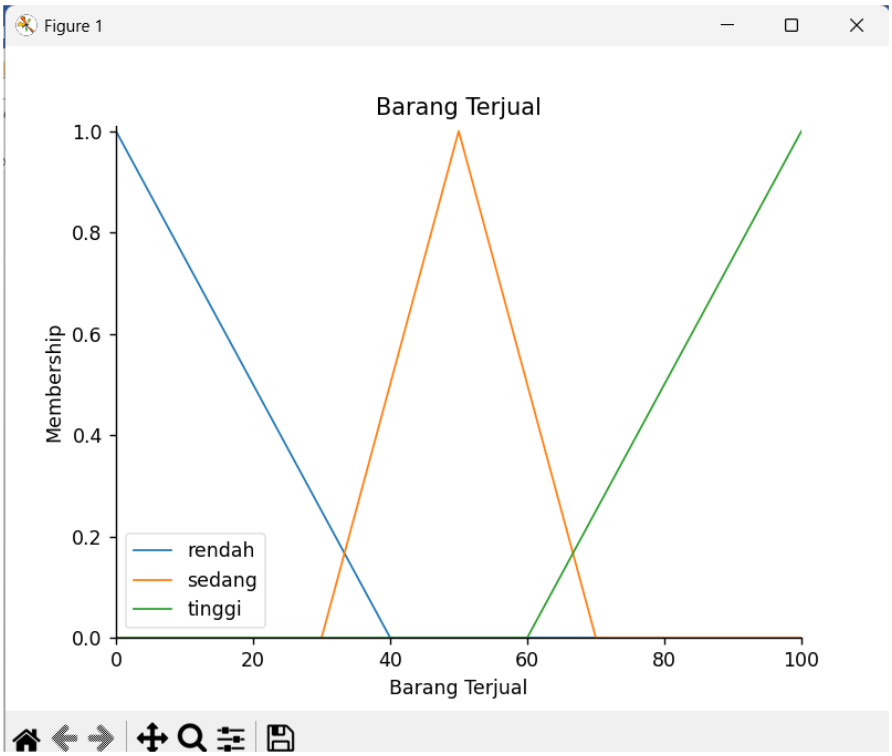
Plt.show() : Menampilkan semua grafik yang sudah dibuat sebelumnya.

OUTPUT

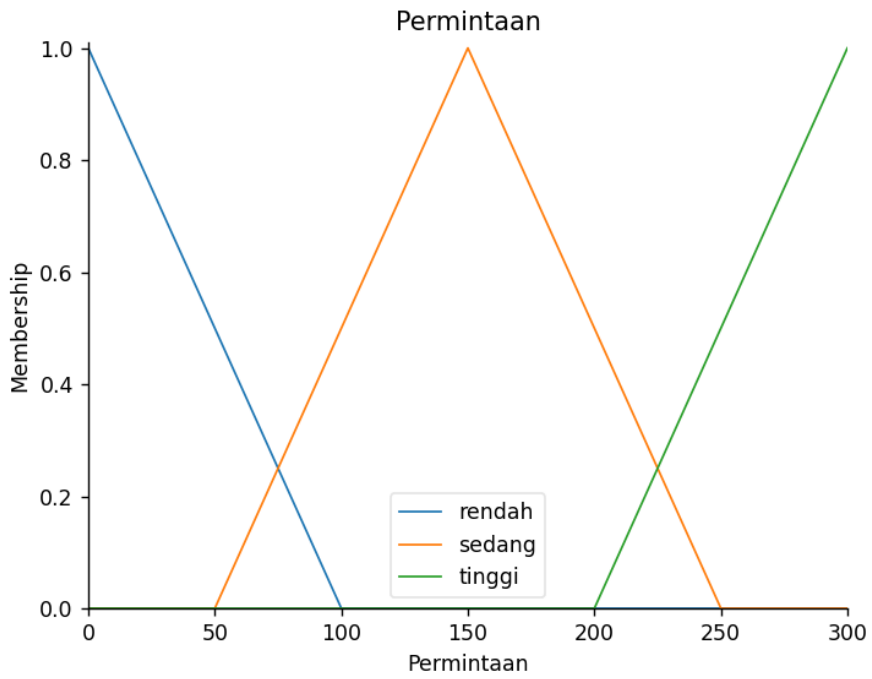
```
PS C:\Users\USER\Pictures\KULIAH\PRAKTIKUM\KECERDASAN BUATAN>
Jumlah stok optimal: 834.6153846153846
```

GRAFIK/FIGURE

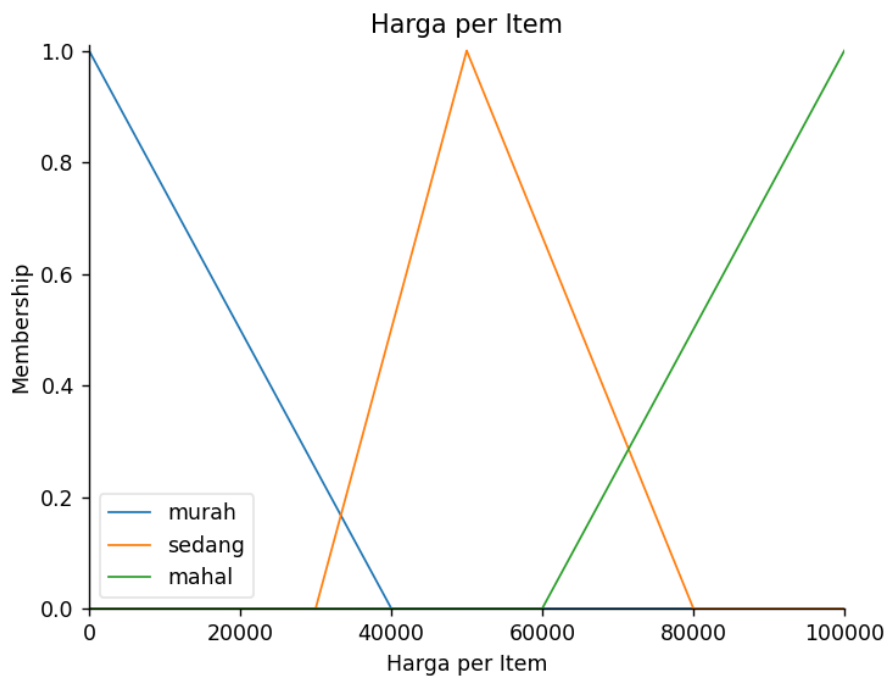
1. Figure 1



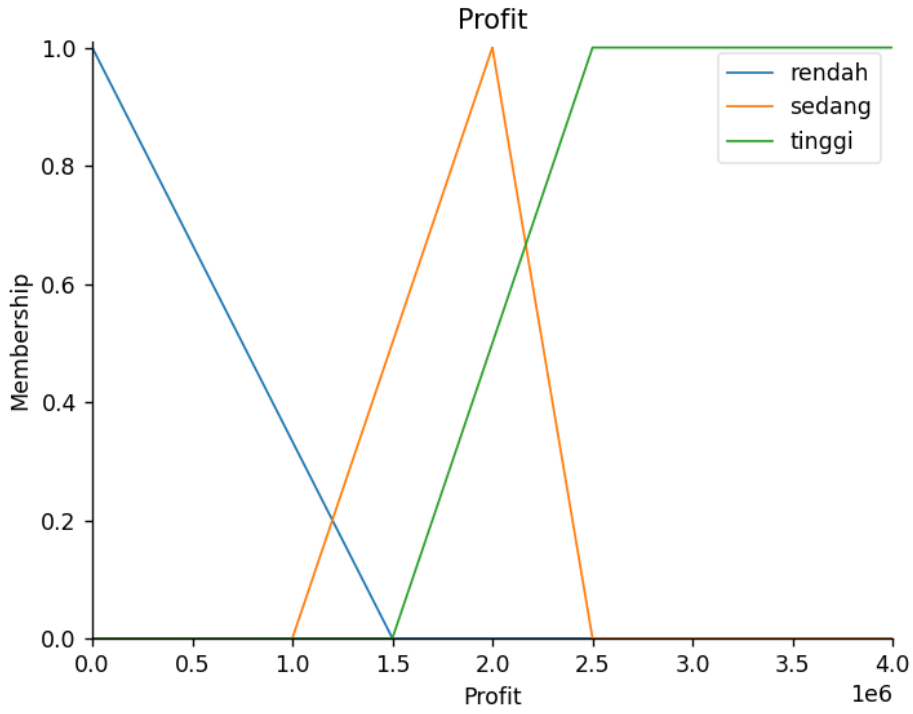
2. Figure 2



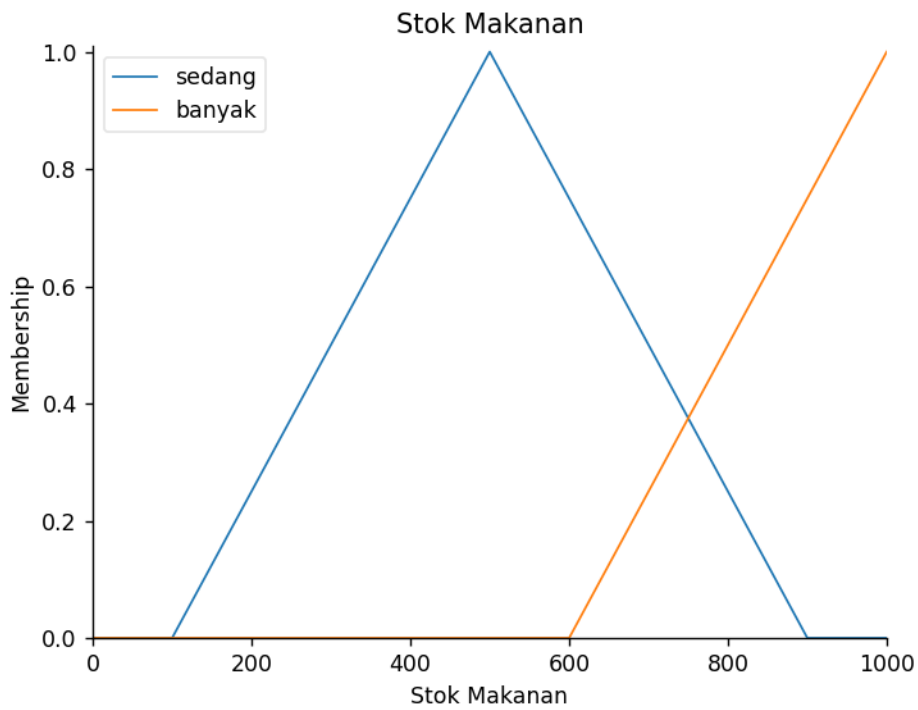
3. Figure 3



4. Figure 4



5. Figure 5



STUDY KASUS 2

- Import

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt
```

Pada bagian awal program, dilakukan proses import library yang akan digunakan untuk membangun sistem fuzzy. Library ini sangat penting karena masing-masing memiliki fungsi yang berbeda dalam proses perhitungan.

a) numpy → membuat array / rentang nilai input-output
b) skfuzzy → membuat logika fuzzy dan membership function
c) control (ctrl) → membangun rule dan sistem fuzzy
d) matplotlib → menampilkan grafik hasil fuzzy

- Variabel

```
# VARIABEL
kejelasan_informasi = ctrl.Antecedent(np.arange(0,101,1),'Kejelasan Informasi')
kejelasan_persyaratan = ctrl.Antecedent(np.arange(0,101,1),'Kejelasan
Persyaratan')
kemampuan_petugas = ctrl.Antecedent(np.arange(0,101,1),'Kemampuan Petugas')
ketersediaan_sarpras = ctrl.Antecedent(np.arange(0,101,1),'Ketersediaan Sarpras')
kepuasan_pelayanan = ctrl.Consequent(np.arange(0,101,1),'Kepuasan Pelayanan')
```

Variabel input:

a) Kejelasan Informasi
b) Kejelasan Persyaratan
c) Kemampuan Petugas
d) Ketersediaan Sarpras

Variabel output:

Kepuasan Pelayanan: Semua variabel menggunakan rentang nilai 0 sampai 100. Rentang ini dipilih karena sesuai dengan skala penilaian pada studi kasus. Sebagai contoh:

a) nilai 0 menunjukkan kondisi sangat buruk
b) nilai 100 menunjukkan kondisi sangat baik

- Membership Function

```
# MEMBERSHIP FUNCTION
for variabel in [kejelasan_informasi, kejelasan_persyaratan, kemampuan_petugas,
ketersediaan_sarpras]:

    variabel['Tidak Memuaskan'] = fuzz.trapmf(variabel.universe,[0,0,30,50])

    variabel['Cukup Memuaskan'] = fuzz.trapmf(variabel.universe,[30,45,65,80])

    variabel['Memuaskan'] = fuzz.trapmf(variabel.universe,[70,85,100,100])
```

Setiap variabel input dibagi menjadi tiga kategori, yaitu:

a) Tidak Memuaskan
b) Cukup Memuaskan
c) Memuaskan

Pemilihan bentuk trapesium bertujuan agar nilai pada suatu rentang memiliki tingkat keanggotaan yang stabil.

- Function Output

```
#output variabel
kepuasan_pelayanan['Tidak Memuaskan'] =
fuzz.trapmf(kepuasan_pelayanan.universe,[0,0,10,25])

kepuasan_pelayanan['Kurang Memuaskan'] =
fuzz.trapmf(kepuasan_pelayanan.universe,[15,25,35,45])

kepuasan_pelayanan['Cukup Memuaskan'] =
fuzz.trapmf(kepuasan_pelayanan.universe,[35,45,60,70])

kepuasan_pelayanan['Memuaskan'] =
fuzz.trapmf(kepuasan_pelayanan.universe,[60,70,80,90])

kepuasan_pelayanan['Sangat Memuaskan'] =
fuzz.trapmf(kepuasan_pelayanan.universe,[80,90,100,100])
```

Output dibagi menjadi lima kategori:

- Tidak Memuaskan
- Kurang Memuaskan
- Cukup Memuaskan
- Memuaskan

- Sangat Memuaskan

Tujuannya agar hasil akhir lebih spesifik dan mudah dianalisis.

- Rules

```
# RULES 1 - 81 (EXCEL)
rule1 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] &
kejelasan_persyaratan['Tidak Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] &
ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Kurang Memuaskan'])
rule2 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] &
kejelasan_persyaratan['Tidak Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] &
ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
rule3 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] &
kejelasan_persyaratan['Tidak Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] &
ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
```

dst… sampai rule 81

Pada program ini terdapat 81 aturan yang ditulis manual. Hal ini karena terdapat 4 variabel input, dan masing-masing memiliki 3 kategori. Sehingga jumlah kombinasi aturan adalah:

$3 \times 3 \times 3 \times 3 = 81$

a) kejelasan informasi memuaskan
b) persyaratan memuaskan
c) kemampuan petugas memuaskan
d) sarpras memuaskan

maka hasilnya adalah: kepuasan pelayanan sangat memuaskan, Semakin baik kombinasi input, maka output yang dihasilkan juga semakin tinggi.

- Sistem Fuzzy

```
# SISTEM FUZZY
sistem_kepuasan = ctrl.ControlSystem([
rule1,rule2,rule3,rule4,rule5,rule6,rule7,rule8,rule9,
rule10,rule11,rule12,rule13,rule14,rule15,rule16,rule17,rule18,
rule19,rule20,rule21,rule22,rule23,rule24,rule25,rule26,rule27,
rule28,rule29,rule30,rule31,rule32,rule33,rule34,rule35,rule36,
rule37,rule38,rule39,rule40,rule41,rule42,rule43,rule44,rule45,
rule46,rule47,rule48,rule49,rule50,rule51,rule52,rule53,rule54,
rule55,rule56,rule57,rule58,rule59,rule60,rule61,rule62,rule63,
rule64,rule65,rule66,rule67,rule68,rule69,rule70,rule71,rule72,
rule73,rule74,rule75,rule76,rule77,rule78,rule79,rule80,rule81,
```

```
])

simulasi = ctrl.ControlSystemSimulation(sistem_kepuasan)
```

Bagian ini berfungsi untuk:

- menggabungkan semua rule
- membangun sistem inferensi fuzzy
- menyiapkan simulasi

bagian ini adalah mesin utama yang akan memproses seluruh logika fuzzy.

- Input

```
#input
simulasi.input['Kejelasan Informasi'] = 80
simulasi.input['Kejelasan Persyaratan'] = 75
simulasi.input['Kemampuan Petugas'] = 85
simulasi.input['Ketersediaan Sarpras'] = 90
```

Data yang digunakan yaitu:

a) Kejelasan Informasi = 80
b) Kejelasan Persyaratan = 75
c) Kemampuan Petugas = 85
d) Ketersediaan Sarpras = 90

Nilai ini akan diproses oleh sistem fuzzy untuk menghasilkan tingkat kepuasan pelayanan.

- Proses (Compute)

```
#proses (compute)
simulasi.compute()
```

Pada bagian ini terdapat tiga tahap utama:

a) Fuzzifikasi: mengubah nilai numerik input menjadi nilai keanggotaan fuzzy
b) Inferensi: menjalankan seluruh 81 rules berdasarkan nilai input
c) Defuzzifikasi: mengubah hasil fuzzy menjadi nilai numerik tunggal

- Output & Visualisasi Akhir

```
#output & visualisasi akhir
hasil = simulasi.output['Kepuasan Pelayanan']
print("Nilai Kepuasan Pelayanan:", hasil)
```

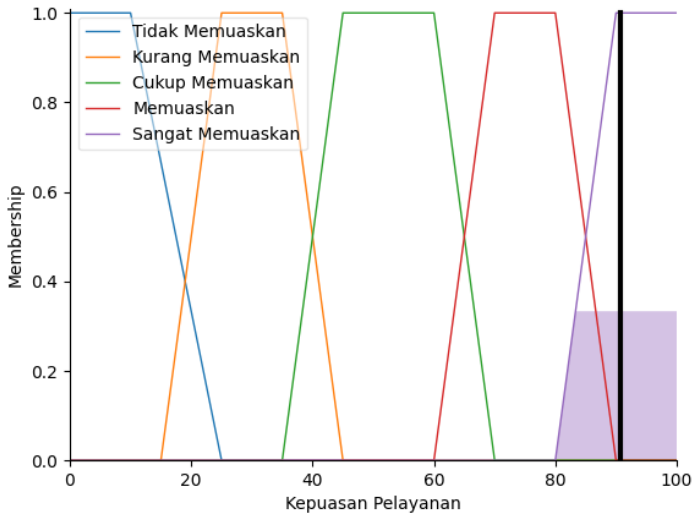
```
kepuasan_pelayanan.view(sim=simulasi)
plt.show()
```

menampilkan hasil akhir dari system fuzzy

HASIL YANG DIPEROLEH:

```
python -u "c:\Users\USER\Pictures\KULIAH\PRAK
N BUATAN\Pertemuan 3\Pelayanan.py"
Nilai Kepuasan Pelayanan: 90.80808080808082
```

GRAFIK



Grafik tersebut menunjukkan hasil akhir dari proses fuzzy pada variabel kepuasan pelayanan, di mana kurva-kurva berwarna merepresentasikan lima kategori yaitu Tidak Memuaskan, Kurang Memuaskan, Cukup Memuaskan, Memuaskan, dan Sangat Memuaskan yang saling tumpang tindih sebagai ciri logika fuzzy. Garis hitam vertikal menunjukkan nilai hasil defuzzifikasi yang berada di sekitar angka 90, yang terletak pada area peralihan antara kategori Memuaskan dan Sangat Memuaskan, namun lebih dominan ke kategori Sangat Memuaskan. Hal ini menunjukkan bahwa berdasarkan input yang diberikan, sistem menilai kualitas pelayanan secara keseluruhan berada pada tingkat sangat baik sehingga menghasilkan tingkat kepuasan yang tinggi.